Shadowing is good for safety

Document number	P2951R1
Date	2023-7-16
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	SG23 Safety and Security Evolution Working Group (EWG) Library Evolution Working Group (LEWG)

Shadowing is good for safety

Table of contents

Changelog

R1

- updated `1st request`, explaining advantage of language solution over library solution in terms of error message
- added `std::optional` example to `2nd request`
- added the `4th request` and its corresponding FAQ
- added a `Safety and Security` section

- revised `Summary` section

Abstract

Removing arbitrary limitations in the language associated with shadowing can improve how programmers deal with invalidation safety issues. It can also benefit a programmer's use of const correctness which impacts immutability and thread safety concerns. Regardless, it promotes simple and succinct code.

Motivational Examples

1st request: It would be beneficial if programmers could shadow a variable with void initialization instead of having to resort to a tag class.

removing names

Present Workaround	Request

`#include <string>`
`#include <vector>`
 	
`using namespace std;`
 	
`struct dename{};// alt: unname, useless`
 	
`int main()`
`{`
`  vector<string> vs{"1", "2", "3"};`
`  for (auto &s : vs) {`
`  dename vs;`
`  // this helps prevents iterator`
`  // and reference invalidation`
`  // as the programmer can't use`
`  // the container being worked on`
`  }`
`}`
	`#include <string>`
	`#include <vector>`
	
	`using namespace std;`
	
	`int main()`
	`{`
	`  vector<string> vs{"1", "2", "3"}`
	`  for (auto &s : vs) {`
	`  void vs;`
	`  // or`
	`  // auto vs;`
	`  // this helps prevents iterato`
	`  // and reference invalidation`
	`  // as the programmer can't use`
	`  // the container being worked`
	`  }`
	`}`

This feature allows programmers to relinquish access to the variable preventing all further operations that could jeopardize safety.

Even if this feature could not be standardized as a language feature, by removing a non breaking restriction, the tag class is adequate, provided the tag class name could be standardized as a library feature.

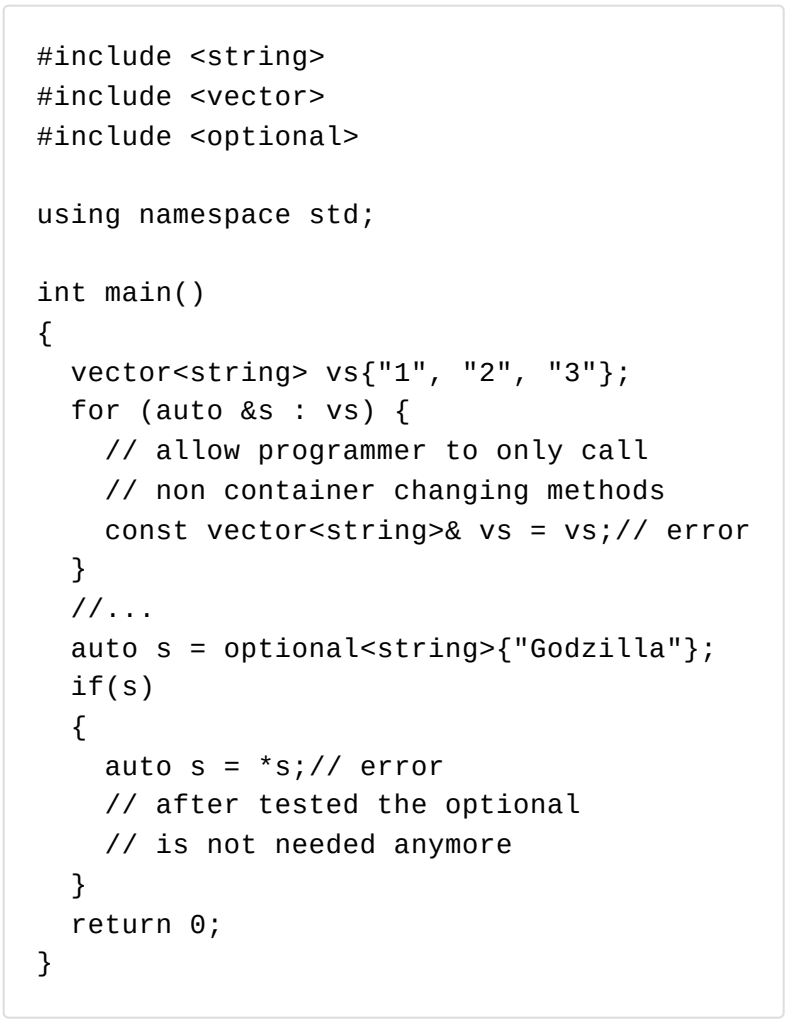
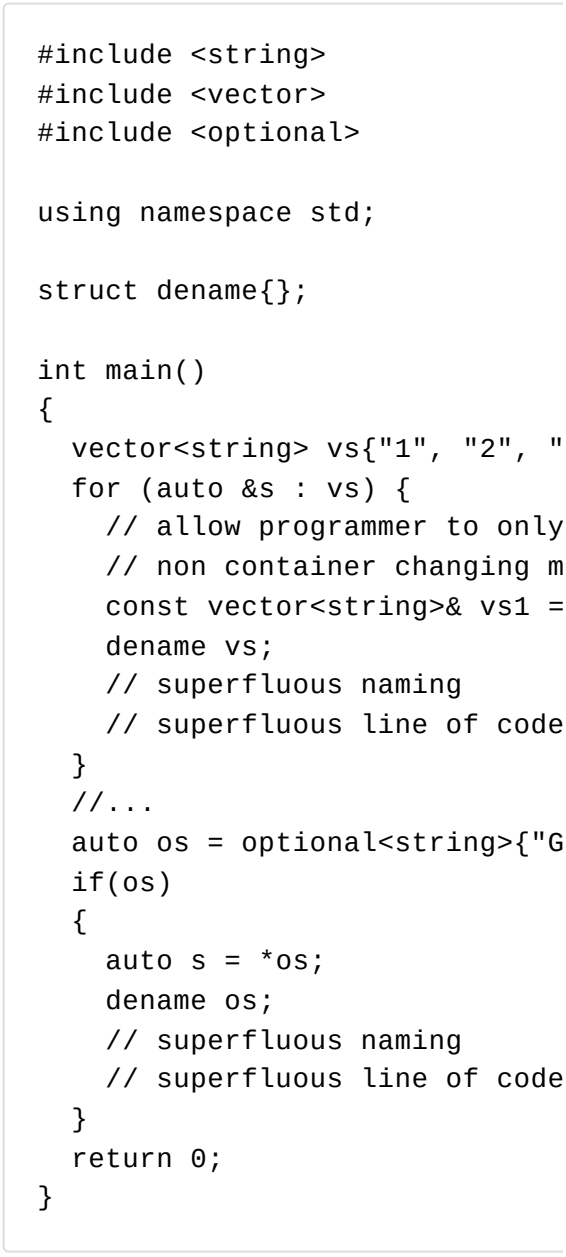
The advantage of having the language feature over the library feature would appear in the error message.

void vs;	error: 'vs' was not declared in this scope
dename vs;	error: 'struct dename' has no member named '*****'

In the language case, `void vs;` , the focus is on the variable that was denamed, which is better. With the library case, `dename vs;` , the focus is on the non existent member.

2nd request: It would be beneficial if programmers could initialize shadowed variables with the variable that is being shadowed.

shadowing limitation: initialization

Present and Request	Present Workaround
	

By temporarily restricting access to non `const` methods, programmers prevent unintended mutations that could jeopardize safety.

Implementation Experience

gcc	this works in gcc
clang	warning: reference ‘vs’ is not yet bound to a value when used within its own initialization [-Wuninitialized]
msvc	warning C4700: uninitialized local variable ‘vs’ used

3rd request: It would be beneficial if programmers could shadow variables without having to involve a child scope.

shadowing limitation: same level

Present and Request	Present Workaround

`#include <string>`
`#include <vector>`
 	
`using namespace std;`
 	
`int main()`
`{`
`  vector<string> vs{"1", "2", "3"};`
`  // done doing complex initializaton`
`  // want it immutable here on out`
`  const vector<string>& vs = vs;// error`
`  return 0;`
`}`
	`#include <string>`
	`#include <vector>`
	
	`using namespace std;`
	
	`struct dename{};`
	
	`int main()`
	`{`
	`  vector<string> vs{"1", "2", "3"`
	`  // done doing complex initializ`
	`  // want it immutable here on ou`
	`  {`
	`  const vector<string>& vs1 = v`
	`  dename vs;`
	`  // superfluous naming`
	`  // superfluous lines of code`
	`  // superfluous level`
	`  }`
	`  // OR use immediately invoked`
	`  // lambda function`
	`  [](const vector<string>& vs)`
	`  {`
	`  // superfluous lines of code`
	`  // superfluous level`
	`  }(vs);`
	`  return 0;`
	`}`

By relinquish access to non `const` methods as quickly as possible, programmers prevent unintended mutations that could jeopardize safety.

The error in the `Present and Request` example may read something like “error: conflicting declaration ‘const vector<string> vs’//note: previous declaration as ‘vector<string> vs’”.

4th request: All of the previous requests have either been hiding variable altogether or replacing with an unconditionally casting. It would be beneficial if programmers had a mechanism for conditional casting.

NEW shadowing feature: conditional casting

Request #4

```
#include <string>
#include <optional>
#include <memory>

using namespace std;

int main()
{
  auto s = optional<string>{"Godzilla"};
  if(s as string)// or if(s is string)
  {
    // s is string&
  }
  else
  {
    // s is still optional<string>
  }
  auto i = shared_ptr<int>{42};
  if(i as string)// or if(i is string)
  {
    // i is int&
  }
  else
  {
    // i is still shared_ptr<int>
  }
  return 0;
}
```

Request #2

```
#include <string>
#include <optional>
#include <memory>

using namespace std;

int main()
{
  auto s = optional<string>{"Godz
  if(s)
  {
    auto s = *s;
    // s is string&
  }
  else
  {
    // s is still optional<string
  }
  auto i = shared_ptr<int>{42};
  if(i)
  {
    auto i = *i;
    // s is int&
  }
  else
  {
    // i is still shared_ptr<int>
  }
  return 0;
}
```

This request is all about being simple and succint. The programmer need not know which conversion, `*`, `get()` or `value()`, gets paired with which test.

This is similar to what Herb Sutter proposed in `Pattern matching using is and as` [1]. While that proposal was focused on pattern matching, this proposal is focused on shadowing and the resulting safety benefits presented.

This functionality exists in the Kotlin [2] and other programming languages.

Safety and Security

Does these requests provide any safety or security?

“In information security, computer science, and other fields, the principle of least privilege (PoLP), also known as the principle of minimal privilege (PoMP) or the principle of least authority (PoLA), requires that in a particular abstraction layer of a computing environment, every module (such as a process, a user, or a program, depending on the subject) must be able to access only the information and resources that are necessary for its legitimate purpose.” [3]

Typically, when someone considers this principle it is in the context that someone is restricting the access of someone or something else. However, it also applies to someone who have been given access, then relinquishing that access or a portion of it when it is no longer needed. The later context is what shadowing is about. When a programmer dename/void their access to a variable they are relinquishing access for the remainder of the scope. When a programmer restrict their access to a variable to only its `const` methods than they relinquish part of their access. In both cases, it means you can’t mutate an instance, which means the defensive programmer proactively avoid iterator, reference and pointer invalidation. Attempting to call a member that one does not have access to results in an error being provided by the compiler instead of by some external tool.

This is also related to borrow checking. Borrow checking is more about aliasing and restricting ownership to just 1. Shadowing allows programmers to reduce their aliases, actually accessible reference count, to 1 just like borrow checking and in some cases even farther to just 0.

Shadowing is of benefit of certain analyzers. Analyzer processing time is usually directly proportional to some number of instances or relationships between instances. For some analyzers, denaming instances could be used to reduces number of concerned instances. Shadowing in these cases would essentially be a analyzer hint.

Summary

Since all of these requests are independent and compatible, the C++ community could adopt any combination of them. This proposal brings all these requests together because they are related.

The advantages of adopting said proposal are as follows:

1. It better allows programmers to use the compiler to avoid, debug and identify iterator, reference and pointer invalidation issues.
2. This proposal aids in const correctness which is good for immutability and thread safety.
3. More shadowing promotes simple and succinct code.

Frequently Asked Questions

Why dename instead of unname?

1. dename [4]: To remove the name from
2. unname [5]: To cease to name; to deprive (someone or something) of their name

While both names would work, dename seems simpler to me than unname. Personally, I think using `void` or slightly less so `auto` would be the more intuitive `C++` name.

Why `as` instead of `is` ?

1. `as` focuses on the casting and hence what a variable will be
2. `is` focuses on the testing and hence what a variable was

Since the `if` clause is more focused on what the variable will be instead of what it was, going with `as` over `is` seemed like a intuitive choice.

References

1. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2392r1.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2392r1.pdf) ↩

2. https://kotlinlang.org/docs/typecasts.html (https://kotlinlang.org/docs/typecasts.html) ↩

3. https://en.wikipedia.org/wiki/Principle_of_least_privilege (https://en.wikipedia.org/wiki/Principle_of_least_privilege) ↩

4. https://en.wiktionary.org/wiki/dename (https://en.wiktionary.org/wiki/dename) ↩

5. https://en.wiktionary.org/wiki/unname (https://en.wiktionary.org/wiki/unname) ↩